

1. Greedy Technique

A greedy algorithm is an algorithm that always tries to find the best solution for each sub problem with the hope that this will yield an optimal solution for the problem as a whole.

Greedy algorithms are simple and straight forward and are easy to design and easy to implement.

Algorithm Greedy (D, n)

1. solution $\leftarrow \emptyset$
2. for $i \leftarrow 1$ to n do
 - {
3. $s \leftarrow \text{select}(D)$
 - if (feasible (solution, s)) then
 - solution $\leftarrow \text{union}(\text{solution}, s)$;
- }

• What is feasible solution and optimal solution?

feasible solution:

The choice made must satisfy the constraint. Any solution that satisfies the given constraint is called a feasible solution.

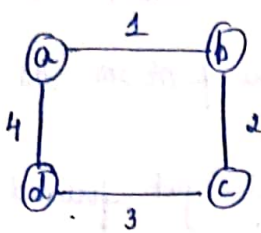
Optimal solution:

The current choice made has to be the best local choice among all feasible solutions.

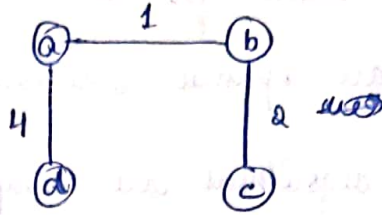
Spanning tree :-

"A spanning tree of a graph 'G' is a subgraph of 'G' in which all the vertices are connected without forming cycles."

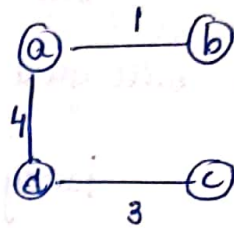
Eg 1:



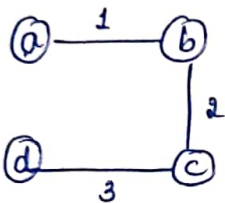
→



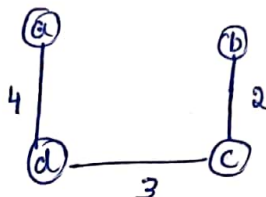
(i)
Cost = 7



(ii) Cost = 8

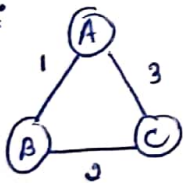


(iii) Cost = 6

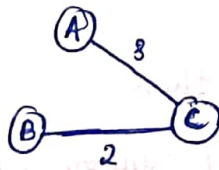


(iv) Cost = 9

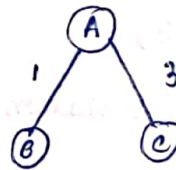
Eg 2:



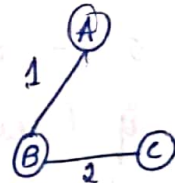
→



(i) Cost = 5



(ii) Cost = 4



(iii) Cost = 3

• Minimum Spanning tree :

A minimum spanning tree of a graph 'G' is a spanning tree whose cost is minimum.

In Eg 1, minimum spanning tree is fig (iii) as it has minimum cost i.e. 6.

• Applications of Spanning tree :

- Designing efficient routing algorithm
- Computer network routing protocol.
- Cluster analysis.

.. Minimum cost spanning tree algorithms :-

.. We have two possible algorithms:

1) Prim's Algorithm

2) Kruskal Algorithm

1) Prim's Algorithm :

i) Initialize the minimum spanning tree with a vertex chosen at random.

ii) Find all the edges that connect the tree to new vertices, find the minimum and add it to the tree.

iii) Keep repeating step (ii) until we get the minimum spanning tree.

Algorithm Prim's (a)

1. $V_T \leftarrow \{v_0\}$

2. $E_T \leftarrow \phi$

3. for $i \leftarrow 1$ to $|V| - 1$ do

4. find minimum weight edge $e^* = (u^*, v^*)$ among all edges (u, v) ,

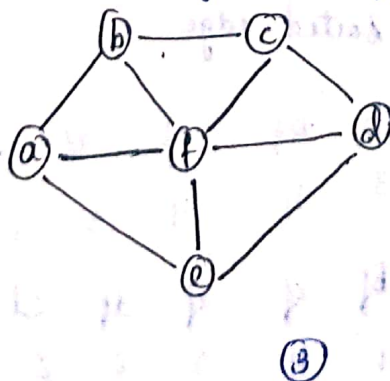
such that $v \in V_T$ & $u \in V - V_T$

5. $V_T \leftarrow V_T \cup \{v^*\}$

6. $E_T \leftarrow E_T \cup \{e^*\}$

7. return E_T

problem: Apply prim's algorithm for the following graph.



Tree Vertices

a(-, -)

b(a, 3)

c(b, 1)

f(c, 4)

e(f, 2)

d(f, 5)

Remaining vertices

b(a, 3), c(a, ∞), d(a, ∞), e(a, 6), f(a, 5)

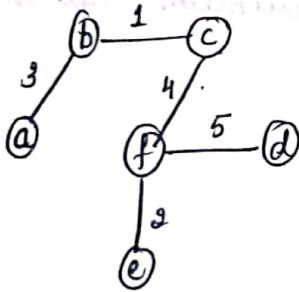
c(b, 1), d(b, ∞), e(a, 6), f(b, 4)

d(c, 6), e(a, 6), f(c, 4)

d(f, 5), e(f, 2)

d(f, 5)

Minimum spanning tree is given by



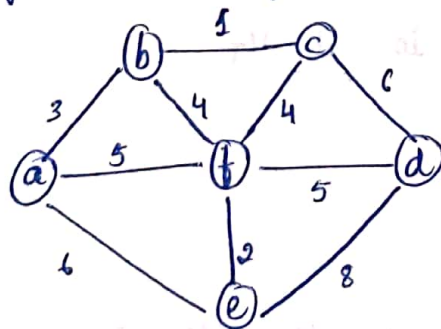
MCST is = 15

• Time Analysis:

- If a graph is weighted matrix and priority queue implemented then running time is $O(|V|^2)$
- If adjacency matrix and priority queue implemented then $O(|E| \log |V|)$.

• Kruskal Algorithm :-

Apply kruskal algorithm for the following graph



Tree edges

-

bc
1

sorted edge

bc	ef	ab	bf	cf	af	df	cd	ae	de
1	2	3	4	4	5	5	6	6	6
ef	ab	bf	cf	af	df	cd	ae	de	
2	3	4	4	5	5	6	6	8	

Tree edges

sorted edges

ef
2

ab	bf	cf	af	df	cd	ae	de
3	4	4	5	5	6	6	8

ab
3

bf	cf	af	df	cd	ae	de
4	4	5	5	6	6	8

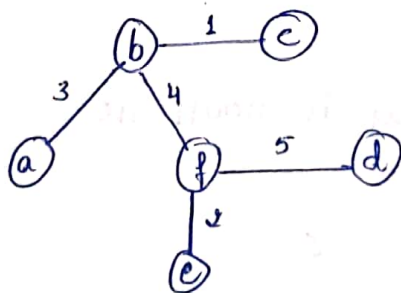
bf
4

cf	af	df	cd	ae	de
4	5	5	6	6	8

df
5

cd	ae	de
6	6	8

Minimum cost spanning tree :



MCST is 15

Algorithm Kruskal (G)

// Constructing MST

// Input: weighted connected graph $G = \langle V, E \rangle$

// Output: E_T - computing MST
Sort E in increasing order of their weights

$E_T \leftarrow \emptyset$

$K \leftarrow 0$

while counter $< |V| - 1$ do

$K \leftarrow K + 1$

if $E_T \cup \{e_k\}$ is acyclic

$E_T \leftarrow E_T \cup \{e_k\}$

counter $\leftarrow$ counter + 1

return E_T

Huffman tree & Huffman code :-

- Huffman's algorithm was developed by David Huffman.
- The algorithm is basically a coding technique for encoding data.
- Huffman's tree is a tree generated from exact freq. as text. Each node has 0 or 2 children and data in leaf node is unique.
- Code generated from Huffman tree is called Huffman code. (left child is indicated by zero and right child is indicated by one)

Huffman's algorithm :

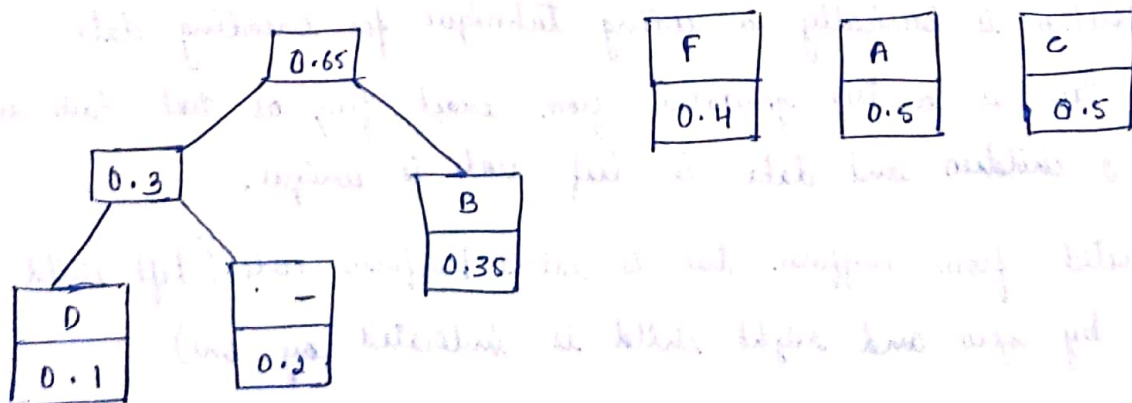
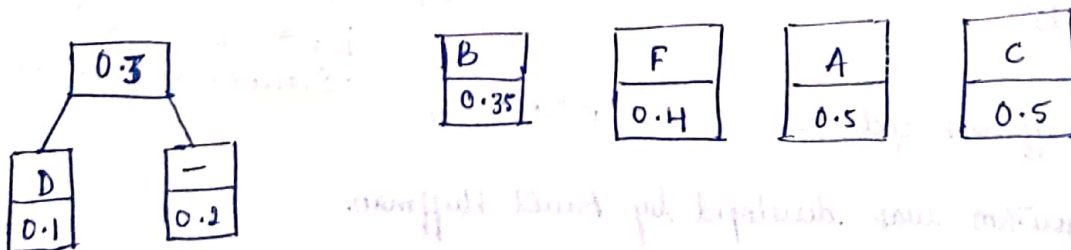
- 1] Initialize n 1-node trees and label them with symbols. Record frequencies of each symbol in its root to indicate weight of tree.
- 2] Find two trees with smaller weights, combine them to form new tree such that smallest represents the left subtree and largest is represented by right subtree. Record sum of their weights in root as new tree.
- Obtain the code by constructing the Huffman tree for the following data

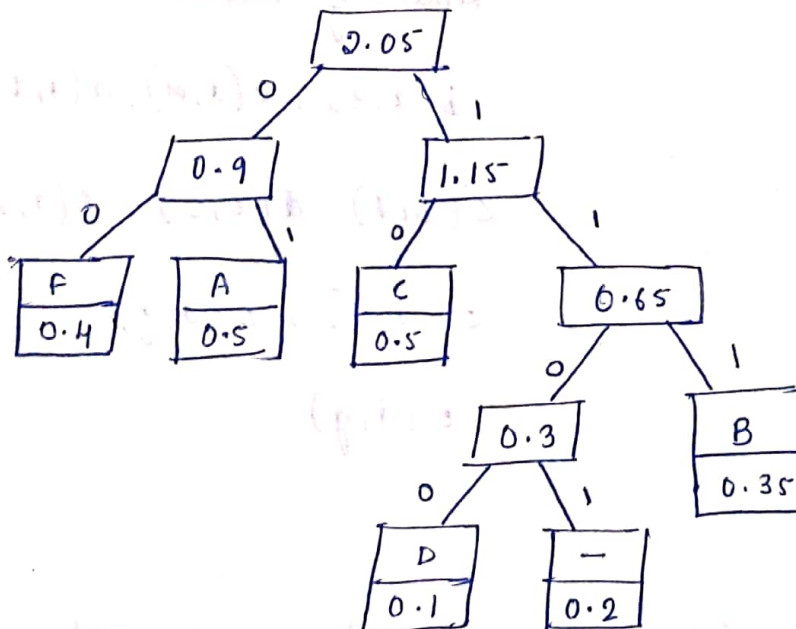
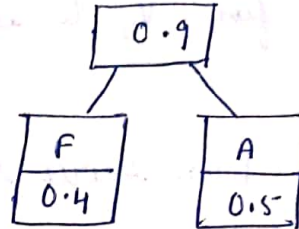
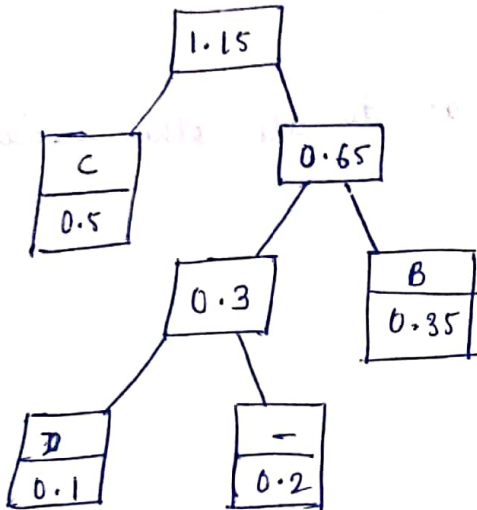
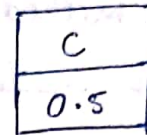
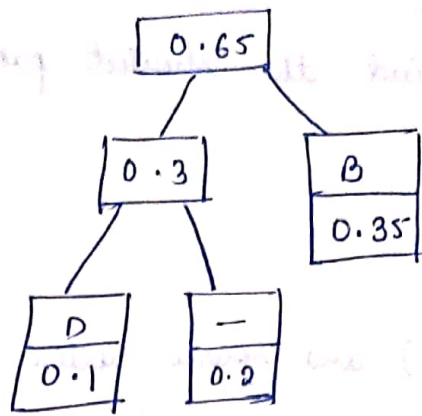
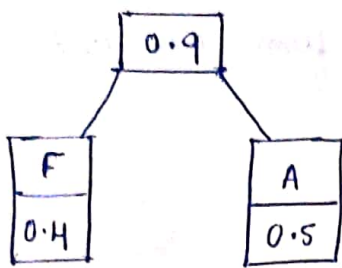
characters	A	B	C	D	F	—
probability	0.5	0.35	0.5	0.1	0.4	0.2

(i) Encode the text DAD — BF

(ii) Decode the text whose encoding is 1100110110

D	—	B	F	A	C
0.1	0.2	0.35	0.4	0.5	0.5





A - 01, B = 111, C - 10, D - 1100

F - 00, - = 1101

DND-BF = 11000111001100111000

1100110110 = D-C

(7)



Single Source Shortest Path:

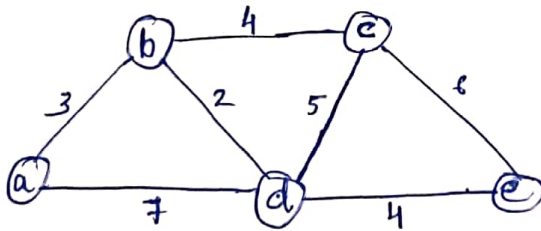
This is the method to find the shortest path from one node to all other nodes.

Step

Dijkstra's Algorithm:

Given graph $G = (V, E)$ and source vertex $V_s \in V$, this algorithm determines shortest path from V_s to every other vertices V_d in V .

- Find the shortest path from source node 'a' to all other nodes using dijkstra's algorithm.



Tree vertex

$a(-10)$

$b(a, 3)$

$d(b, 5)$

$c(b, 7)$

$e(a, 9)$

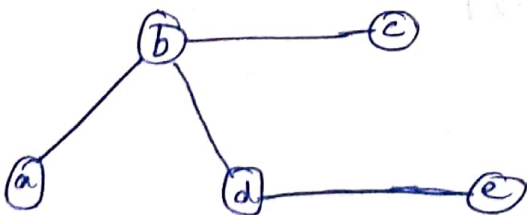
Remaining vertices

$b(a, 3), c(a, \infty), d(a, 7), e(a, \infty)$

$c(b, 7), d(b, 5), e(a, \infty)$

$c(b, 7), e(d, 9)$

$e(d, 9)$



Distance b/w nodes are

$a \rightarrow b = 3$

$a \rightarrow b \rightarrow c = 7$

$a \rightarrow b \rightarrow d = 5$

$a \rightarrow b \rightarrow d \rightarrow e = 9$

• Algorithm Dijkstra (a, s) :-

for every vertex v in V

$d_v \leftarrow \infty$

$P_v \leftarrow \text{null}$

insert (Q, v, d_v)

$d_s \leftarrow 0$

decrease (Q, s, d_s)

$V_T \leftarrow Q$

for $i \leftarrow 0$ to $|V| - 1$ do

$u^* \leftarrow \text{determine}(Q)$

$V_T \leftarrow V_T \cup \{u^*\}$

for every vertex u in $V - V_T$

if $d_{u^*} + w(u^*, u) < d_u$

$d_u \leftarrow d_{u^*} + w(u^*, u)$

$P_u \leftarrow u^*$

decrease (Q, u, d_u)

Backtracking

By,

Prithviraj
Dept. of CSE

In this the problem can be categorized into 3 categories

1) Finding whether there is any feasible solution? - is the decision problem.

2) What is the best solution? - is the optimization problem.

3) Listing of all feasible solutions - is the enumeration problem.

- Backtracking algorithm determines the solution by systematically searching the solution space for the given problem.

Applications:

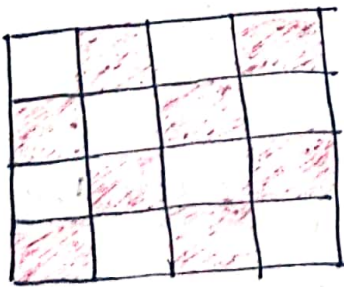
- 8 queen's problem
- Sum of subset problem
- Graph coloring problem
- Finding Hamilton cycles
- Knapsack problem

DEFN: "Backtracking is the process of solving the problem by returning to a previous state in a computation in order to try an alternative strategy."

N-Queens Problem:-

- Consider $n \times n$ chessboard on which we have to place n queens so that no two queens attack each other by being in a same row or same column or on the same diagonal.

eg: 4×4 board



Note: 2-Queens problem is not solvable - can't place 2 queens at different place.

①

problem: solve 4-queens problem (4x4)

q ₁			
		q ₂	

next q₃?

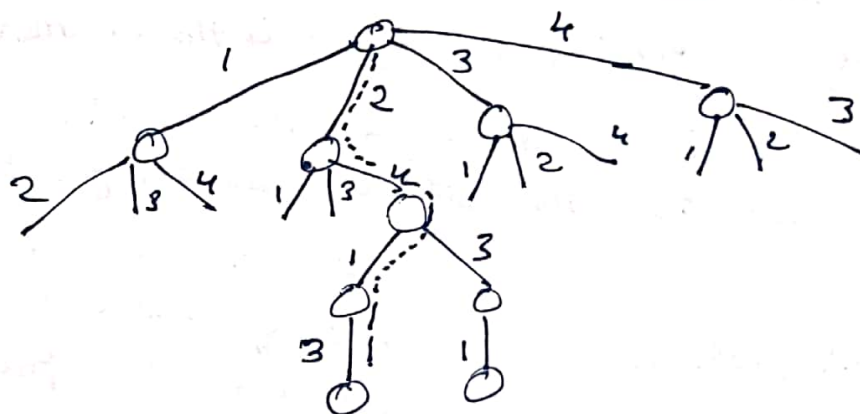
q ₁			
			q ₂
	q ₃		
.	.	.	.

next q₄?

	q ₁		
			q ₂
q ₃			
		q ₄	

solution = (2, 4, 1, 3)

State space tree for 4-queens problem



8-queens problem:-

	1	2	3	4	5	6	7	8
1			q ₁					
2						q ₂		
3		q ₃						
4							q ₄	
5	q ₅							
6				q ₆				
7								q ₇
8					q ₈			

1 solution is

(3, 6, 2, 7, 1, 4, 8, 5)

There are different other solutions also

obtain 2 solutions for 4-queens problem.

	1	2	3	4
1		q ₁		
2				q ₂
3	q ₃			
4			q ₄	

	1	2	3	4
1			q ₁	
2	q ₂			
3				q ₃
4		q ₄		

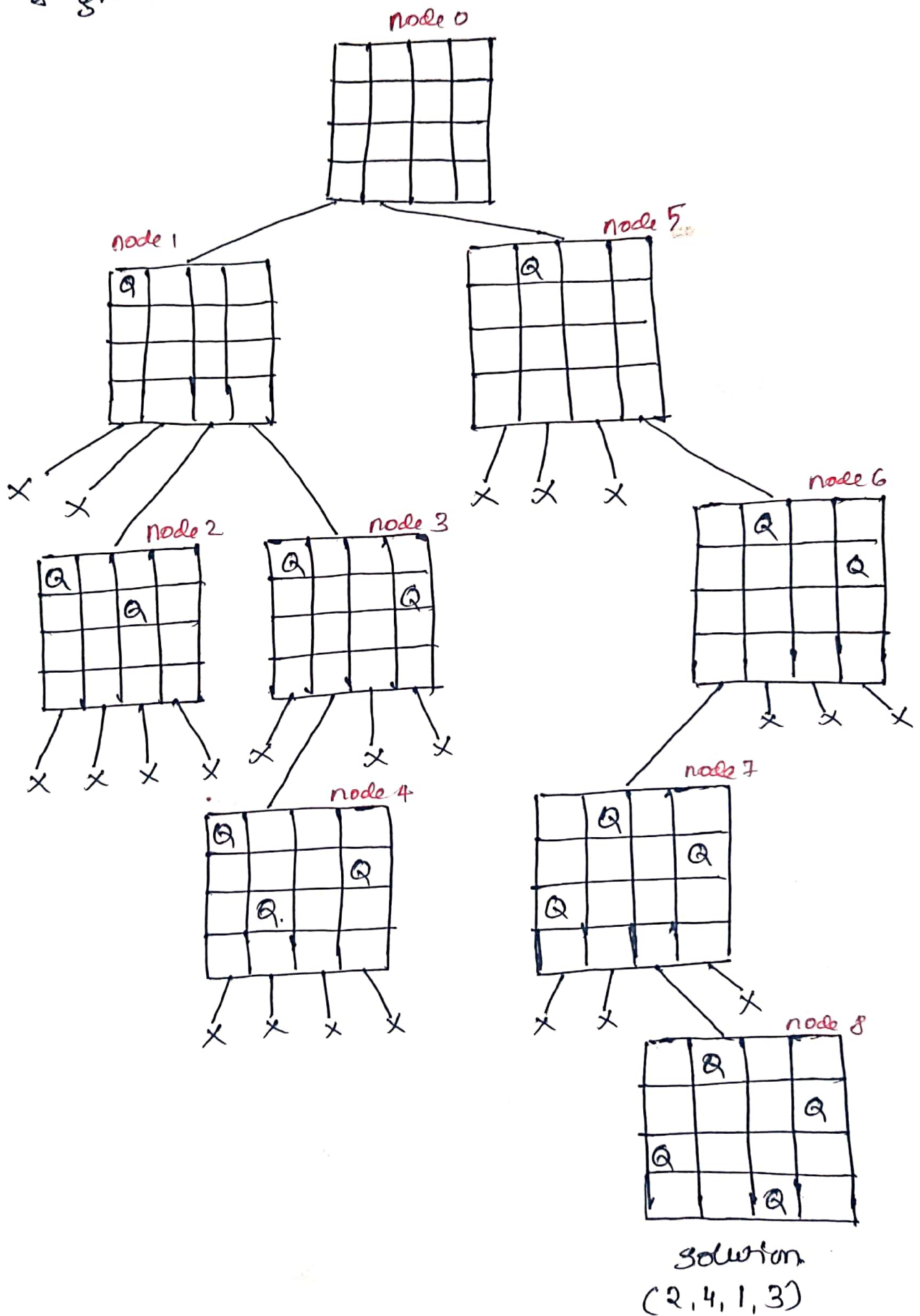
(2, 4, 1, 3)

and

(2, 1, 4, 2)



The state space tree of one possible solution is given below.



Generate 3-solutions for 5-queen problem.

q_1				
		q_2		
				q_3
	q_4			
			q_5	

(1, 3, 5, 2, 4)

	q_1			
			q_2	
q_3				
		q_4		
				q_5

(2, 4, 1, 3, 5)

	q_1			
				q_2
		q_3		
q_4				
			q_5	

(2, 5, 3, 1, 4)

Write an algorithm for backtracking

Algorithm Backtrack()

```

for each  $a_k$  that belongs to  $T(a_1, a_2, \dots, a_{k-1})$  do
{
    if ( $B_k(a_1, a_2, \dots, a_k) = \text{true}$ ) then
    {
        if ( $(a_1, a_2, \dots, a_k)$  is a path to answer node then
            print ( $a[1], a[2], \dots, a[k]$ );
            if ( $k < n$ ) then
                Backtrack( $k+1$ );
        }
    }
}

```

Problems on n-queens: Solve 8-queens problem for a feasible sequence (6, 4, 7, 1)

Sum of Subsets Problem:-

Problem Statement - Let $S = \{s_1, s_2, \dots, s_n\}$ be a set of 'n' positive integers, then we have to find a subset of whole sum is equal to given positive integer 'd'.

It is always convenient to sort the set's elements in ascending order

$$s_1 \leq s_2 \leq \dots \leq s_n$$

Algorithm

Step 1: Start with an empty set

Step 2: Add to the subset, the next element from the list.

(2)

Step 3: If the subset is having sum d then stop with that subset as solution

Step 4: If the subset is not feasible or if we have reached the end of the set then backtrack through the subset until we find the next suitable value.

Step 5: If we have visited all the elements without finding a suitable subset & if no backtracking is possible then stop without solution.

Problem 1: Apply backtracking problem to solve the instance of the sum of subset problem. Also draw state space tree.

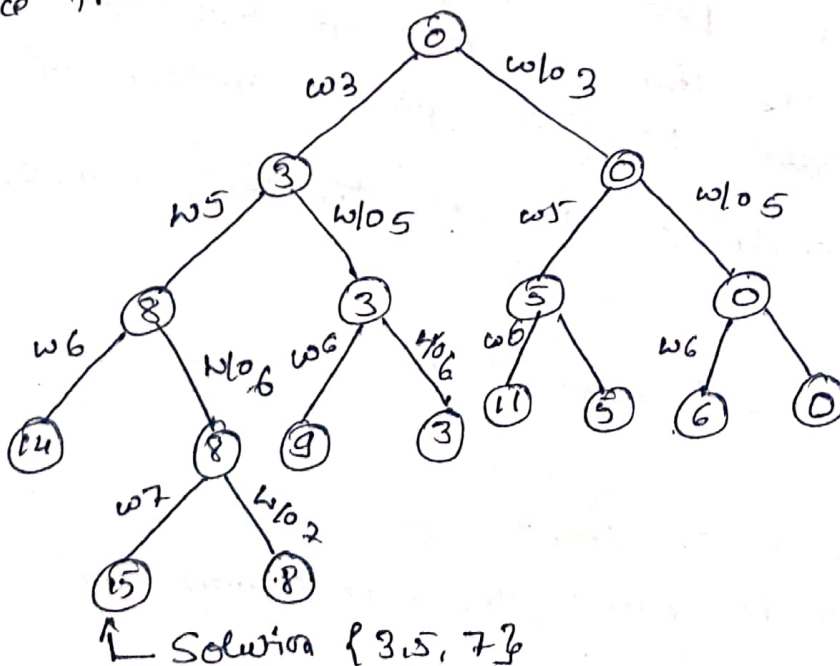
$S = \{3, 5, 6, 7\}$ and $d = 15$

Initially subset = $\{\}$ sum = 0

subset	sum	sum < d
3	3	$3 < 15$
3, 5	8	$8 < 15$
3, 5, 6	14	$14 < 15$
3, 5, 6, 7	21	$21 > 15$
3, 5, 7	15	$15 = 15$
3, 7	10	$10 < 15$

Remarks
next element
— 11 —
— 11 —
Backtrack
solution

state space tree:



2) Consider a set $S = \{5, 10, 12, 13, 15, 18\}$ and $d = 30$. Solve it for obtaining sum of subsets

→ initially subset = $\{\}$ sum = 0

subset	sum	sum < d	Remarks
5	5	$5 < 30$	next element
5, 10	15	$15 < 30$	next element
5, 10, 12	27	$27 < 30$	— " —
5, 10, 12, 13	40	$40 > 30$	Backtrack. (Remove 13)
5, 10, 12, 15	42	$42 > 30$	Back track. (Remove 15)
5, 10, 12, 18	45	$45 > 30$	backtrack (Remove 18 & 13)
5, 10, 13	28	$28 < 30$	next element
5, 10, 13, 15	33	$33 > 30$	backtrack (Remove 15 & 13)
5, 10	30	$30 = 30$	Solution obtained

3) What is backtracking? Apply backtracking algorithm to solve the instance of $S = \{1, 3, 4, 5\}$ and $d = 11$

4) Apply backtracking problem to solve the instance of the sum of subset problem $S = \{1, 2, 5, 6, 8\}$ and $d = 9$.

3. Branch and Bound

In branch and bound method a space tree with all possible solution is generated. Then partitioning or branching is done at each node of the tree.

In the branch & bound technique a set of feasible solutions are partitioned & the subset that do not have optimal solutions are deleted from further considerations. Thus the branch & bound approach has 2 major steps - branching & bounding. These two steps are repeated till the problem is solved.

Branching is the first step of the branch & bound technique, which involves division of a given problem into two or more subproblems. These subproblems are independent problems.

Bounding is the second step, in this step the best solution of the subproblems are identified and used as a lower bound of the given problem. A lower bound is the optimistic estimate of the best solution. differential blue backtracking & Branch and bound

Backtracking

→ solution is traced using DFS

→ Decision problems can be solved using backtracking

→ worst choices can be made while solving problem

Branch & Bound

Here solution can be solved using DFS or BFS

Optimization problems can be solved using branch & bound.

It proceeds on better solution, hence there cannot be a wrong choice

→ State space tree is searched until the solution is obtained.

Optimal solution can be anywhere within state space tree, hence need to be searched completely.

→ Eg: Knapsack, sum of subset, Hamiltonian cycle.

Eg: Job sequencing, TSP, Assignment problems.

Knapsack Problem:-

The procedure for solving the knapsack problem using the branch & bound technique is as follows

Step 1:- Arrange the items that $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$.

Step 2:- Construct a state space tree as a binary tree. Take a node, branching to the left indicates that the item is included and branching to the right indicates that item is excluded.

Step 3:- Compute the upper bound as

$$U_b = v_i + (M - w_i) \times \frac{v_{i+1}}{w_{i+1}}$$

M - Capacity of knapsack

w - weight

v_i - value of i th item

w_i - weight of i th item

i) Solve the following knapsack problem

<u>items</u>	<u>w_i</u>	<u>v_i</u>
1	2	8
2	3	6
3	2	4

Capacity = 6, CM

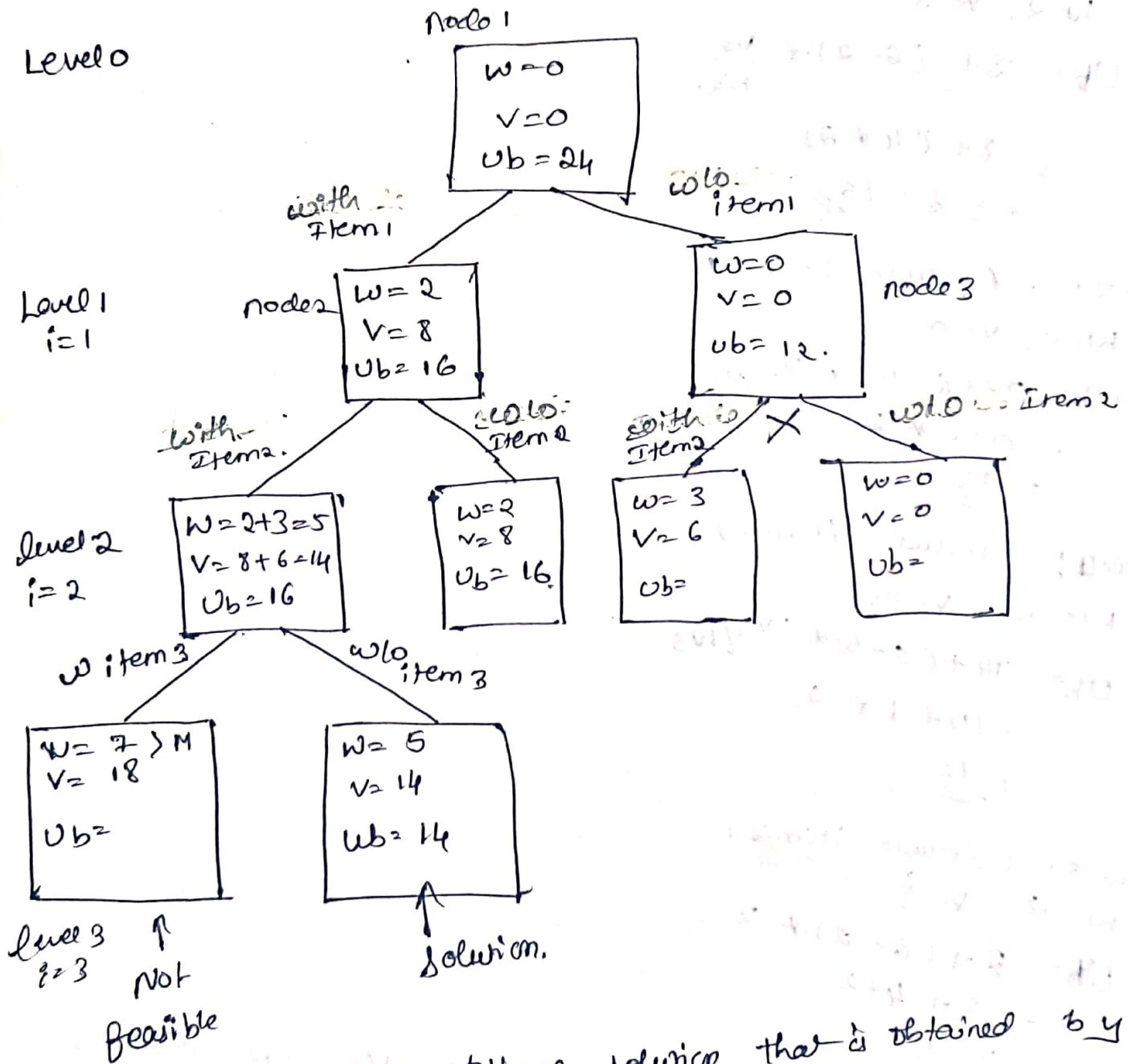
Let us calculate ratio (V_i/W_i)

$$v_1/\omega_1 = 8/2 = 4 ; \quad v_2/\omega_2 = 6/3 = 2 ; \quad v_3/\omega_3 = 4/2 = 2$$

Arrange the items

<u>items</u>	<u>w_i</u>	<u>v_i</u>	<u>$v_i w_i$</u>
1	2	8	4
2	3	6	2
3	2	4	2

Level 0



feasible
 $U_b = 14$ is the optimal solution that is obtained by
 considering with item 1, with item 2, without item 3

Node 1: $W=0, V=0$

$$\begin{aligned} U_b &= V + (M-W) * \frac{V_{i+1}}{W_{i+1}} \\ &= 0 + (6-0) * \frac{V_1}{W_1} \\ &= 6 * 4 = \underline{24} \end{aligned}$$

$$\left[\because \frac{V_{0+1}}{W_{0+1}} \right]$$

node 2: (Include item 1)

$$W=2, V=8$$

$$\begin{aligned} U_b &= 8 + (6-2) * \frac{V_2}{W_2} \\ &= 8 + (4 * 2) \\ &= 8 + 8 = 16 \end{aligned}$$

node 3: (Exclude item 1)

$$W=0, V=0$$

$$\begin{aligned} U_b &= 0 + (6-0) * \frac{V_2}{W_2} \\ &= 0 + 6 * 2 \\ &= \underline{12} \end{aligned}$$

Node 4: (Include item 2)

$$W=5, V=14$$

$$\begin{aligned} U_b &= 14 + (6-5) * \frac{V_3}{W_3} \\ &= 14 + 1 * 2 \\ &= \underline{16} \end{aligned}$$

Node 5: (Exclude item 2)

$$W=2, V=8$$

$$\begin{aligned} U_b &= 8 + (6-2) * 2 \\ &= 8 + 4 * 2 \\ &= \underline{16} \end{aligned}$$

Since in node 4 and node 5 we got the same answer
Hence we can choose any one value. Let us select the
value from node 5.

Node 6: (Include item 3)

$$W = 5 + 2 = 7, \quad V = 14 + 4 = 18$$

$$Ub = 18 + (6 - 7) \times 0.$$

$$W = 7 > M.$$

Hence it is not a feasible solution

Node 7: (Exclude item 3)

$$W = 2 + 3 = 5, \quad V = 14$$

$$Ub = V + (M - W) \times \frac{V_{i+1}}{W_{i+1}}$$

$$Ub = 14 + (6 - 5) \times 0 \quad (\because v_4/v_5 = 0)$$

$$Ub = \underline{14}$$

Check out the other possibility & check out the feasible solution.

Let us understand the use of backtracking in the next example
 Problem 2: Apply 0/1 Knapsack for the following using branch & bound technique, having capacity $W = 16$.

item	weight	profit
1	10	100
2	7	63
3	8	56
4	4	12

→ Find out Profit/Weight ratio (V/W) & sort the list of items in descending order of their ratio

item	weight	Profit	V/W
1	10	100	10
2	7	63	9
3	8	56	7
4	4	12	3

Formula:

$$U_b = V + (M - w) * \frac{V_{i+1}}{W_{i+1}}$$

node 0: without having any item $w=0, v=0$

$$U_b = 0 + (16 - 0) * \frac{V_1}{W_1}$$

$$U_b = 0 + 16 * 10$$

$$U_b = 160$$

node 1: Choosing item 1, $V=100, w=10$

$$U_b = 100 + (16 - 10) * \frac{V_2}{W_2}$$

$$= 100 + (6 * 9)$$

$$= 154$$

node 2: without choosing item 1, $V=0, w=0$

$$U_b = 0 + (16 - 0) * 9$$

$$= 144$$

Compare both upper bound values & select the optimal one (highest)

node 3: Choosing item 2, $w = 10 + 7 = 17$
 $V = 100 + 63 = 163$

$$U_b = 163 + (16 - 17) * \frac{V_3}{W_3}$$

Since $w > M$ i.e. $17 > 16$ this is non-feasible.

node 4: without choosing item 2, $w=10, v=100$

$$U_b = 100 + (16 - 10) * 7$$

$$= 100 + 42$$

$$= 142$$

Note: Compare U_b value of node 4 with previous U_b value of node 2, if that is greater than the present node 'backtrack' to previous node & continue

node 5: choosing item 2, $w=7, v=63$

$$U_b = 63 + (16-7) * \frac{v_3}{w_3}$$

$$= 63 + (9 * 7)$$

$$= 126$$

node 6: without choosing item 2, $w=0, v=0$

$$U_b = 0 + (16-0) * 7$$

$$= 112$$

U_b value in node 5 is greater than node 6 &
Now compare node 5 with node 4 i.e. $142 > 126$ &
hence select node 4 and continue

node 7: choosing item 3 $w=10+8 > M$ This is
not feasible since it exceeds the overall capacity

node 8: without choosing item 3 $w=10, v=100$

$$U_b = 100 + (16-10) * \frac{v_4}{w_4}$$

$$= 100 + 6 * 3$$

$$= 118$$

compare $U_b=118$ with bound value of node 5 and
select accordingly i.e. $126 > 118$ hence continue from

node 5

node 9: choosing item 3, $w=7+8=15$
 $v=63+56=119$

$$U_b = 119 + (16-15) * \frac{v_4}{w_4}$$

$$= 119 + (1 * 3)$$

$$= 122$$

node 10: without choosing item 3 $w=7, v=63$

$$\begin{aligned}U_b &= 63 + (16-7) \times 3 \\&= 63 + 27 \\&= 90\end{aligned}$$

Comparing node 9 and node 10, node 9 has greater upper bound value and hence select node 9 and also compare node 9 with node 8, In both cases value 122 of node 9 is greater & hence proceed with node 9.

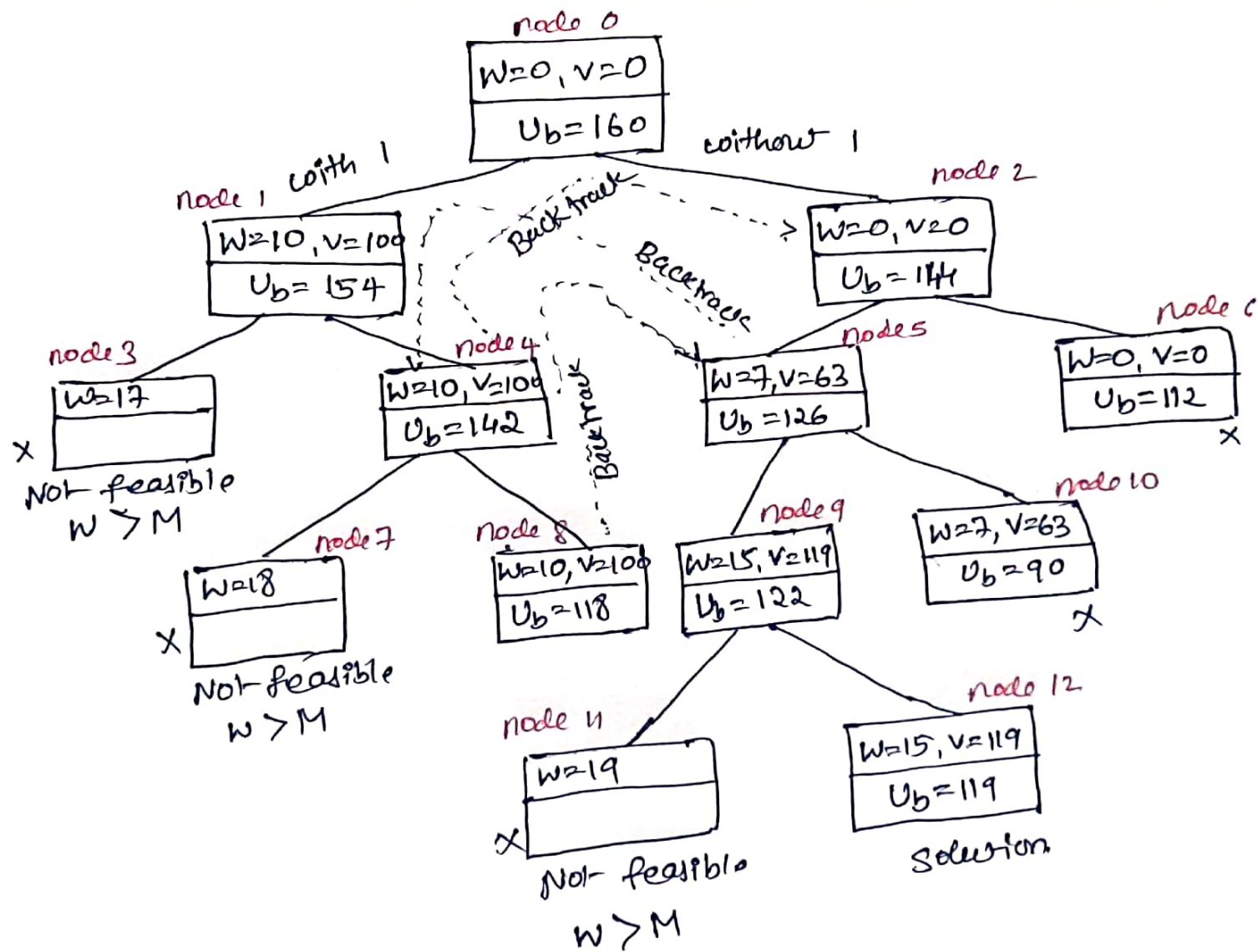
node 11: Choosing item 4: $w=15+4=19 > M$ and hence not feasible

node 12: without choosing item 4 $w=15, v=119$

$$U_b = 119 + (16-15) \times 0 \quad [\because v_5/w_5 = 0]$$

$$U_b = 119$$

We have completed with all items and final optimal solution is 119. This is the maximum profit achieved by selecting item 2 and item 3. Related state space diagram is given as follows



Selected items = {item 2, item 3}

Optimal solution = 119.

Branch and Bound

Assignment Problem:

It is the problem of assigning 'n' people to 'n' jobs so that the total cost of the assignment is as small as possible.

Problem: Apply assignment problem for the following.

$$C = \begin{matrix} & \text{Job1} & \text{Job2} & \text{Job3} & \text{Job4} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix} \end{matrix}$$

It is clear that the cost of any solution including the optimal one cannot be smaller than the sum of the smallest elements in each of the matrix's row.

Here sum is: $2+3+1+4=10$. It is just a lower bound on the cost of any legitimate selection.

Construct a state space tree

$$\begin{matrix} & j_1 & j_2 & j_3 & j_4 \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} 9 \\ 3 \\ 1 \\ 4 \end{matrix}$$

$$lb = 9+3+1+4 = 17$$

$$\begin{matrix} & j_1 & j_2 & j_3 & j_4 \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 9 & 2 & 7 & 8 \\ 6 & 4 & 3 & 7 \\ 5 & 8 & 1 & 8 \\ 7 & 6 & 9 & 4 \end{bmatrix} \end{matrix} \rightarrow \begin{matrix} 2 \\ 3 \\ 1 \\ 4 \end{matrix}$$

$$lb = 2+3+1+4 = 10$$

	j_1	j_2	j_3	j_4
a	9	2	7	8
b	6	4	3	7
c	5	8	1	8
d	7	6	9	4

$$lb = 7 + 4 + 5 + 4 = 20$$

	j_1	j_2	j_3	j_4
a	9	2	7	8
b	6	4	3	7
c	5	8	1	8
d	7	6	9	4

$$lb = 8 + 3 + 1 + 6 = 18$$

lower Bound value with 10 will be chosen i.e. assigning job 2 to person 'a'. Let us start assigning jobs to person 'b'.

	j_1	j_2	j_3	j_4
a		2		
b	6		3	7
c	5		1	8
d	7		9	4

$$lb = 2 + 6 + 1 + 4 = 13$$

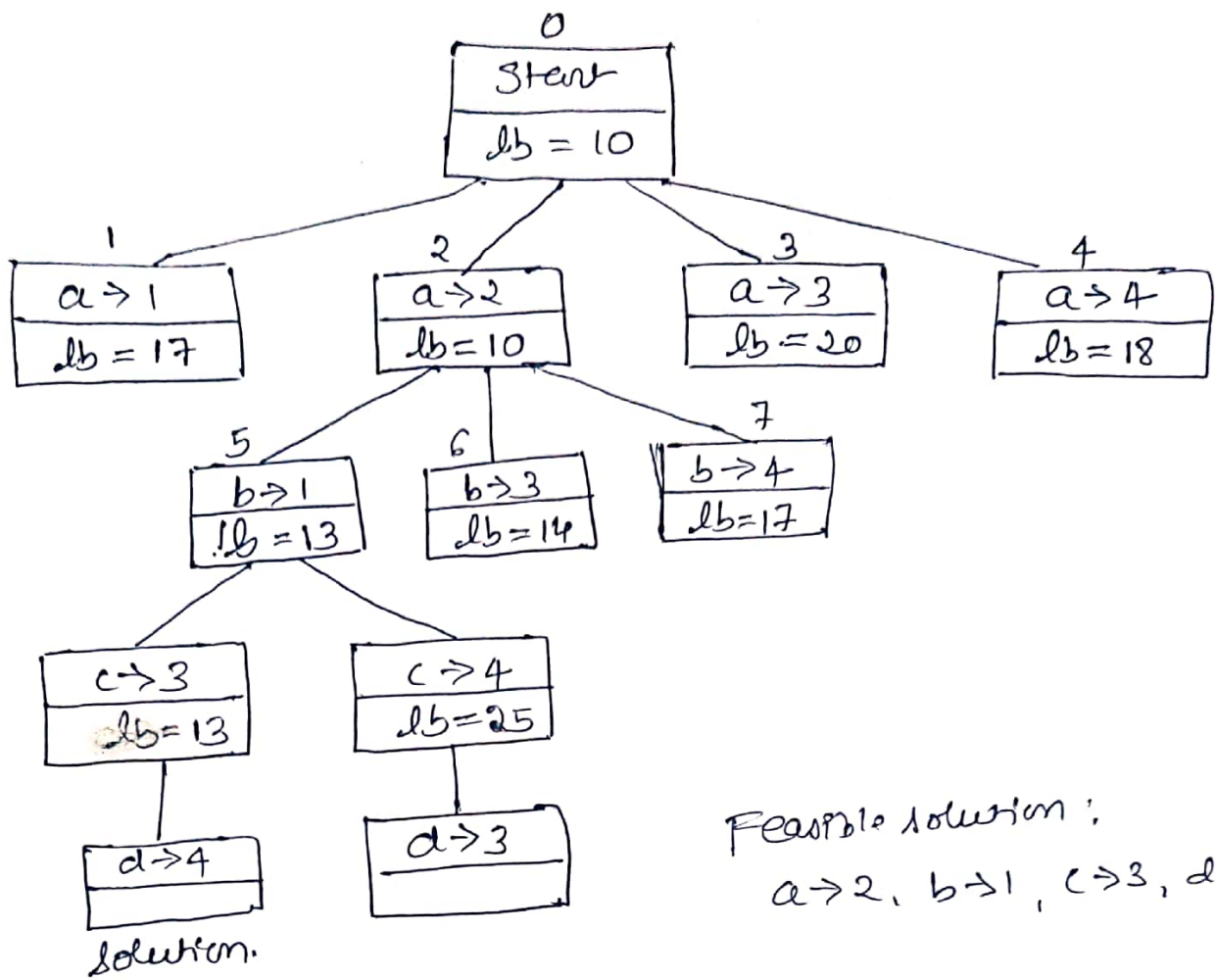
	j_1	j_2	j_3	j_4
a		2		
b	6		3	7
c	5		1	8
d	7		9	4

$$lb = 2 + 3 + 5 + 4 = 14$$

	j_1	j_2	j_3	j_4
a		2		
b	6		3	7
c	5		1	8
d	7		9	4

$$lb = 2 + 7 + 1 + 7 = 17$$

lower Bound = 13 is the least value. to be chosen



	j ₁	j ₂	j ₃	j ₄	
a		2			→ 2
b	6				→ 6
c			1	8	→ 1
d			9	4	→ 4

	j ₁	j ₂	j ₃	j ₄	
a		2			→ 2
b	6				→ 6
c				8	→ 8
d			9	4	→ 9

$$lb = 2 + 6 + 1 + 4 = 13$$

$$lb = 2 + 6 + 8 + 9 = 25$$

lb = 13 is the lowest value.

Finally assign job 4 to d.

Chapter 4

P, IVP, and IVP-complete Problems

We say that an algorithm solves a problem in polynomial time if its worst-case time efficiency belongs to $O(p(n))$ where $p(n)$ is a polynomial of the problem's input size n . Problems that can be solved in polynomial time are called tractable, problems that cannot be solved in polynomial time are called intractable.

For many of the problems we know and study, the best algorithms for their solution have computing times can be clustered into two groups;

1. Solutions are bounded by the **polynomial-**

Examples include Binary search $O(\log n)$, Linear search $O(n)$, sorting algorithms like merge sort $O(n \log n)$, Bubble sort $O(n^2)$ & matrix multiplication $O(n^3)$ or in general $O(n^k)$ where k is a constant.

2. Solutions are bounded by a **non-polynomial-**

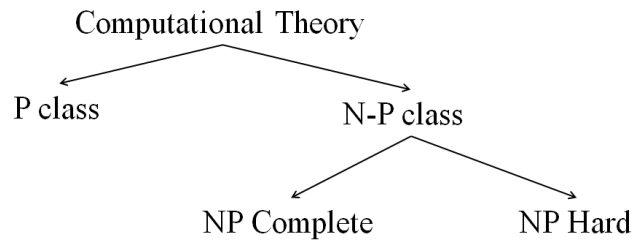
Examples include travelling salesman problem $O(n^2 2^n)$ & knapsack problem $O(2^{n/2})$. As the time increases exponentially, even moderate size problems cannot be solved.

Several reasons for drawing the intractability line are:

1. In the following table entry, it implies that we cannot solve arbitrary instances of intractable problems in a reasonable amount of time unless such instances are very small.

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

2. There might be a huge difference between the running times in $O(p(n))$ for polynomials of drastically different degrees, there are very few useful polynomial-time algorithms with the degree of a polynomial higher than three.
3. Polynomial functions possess many convenient properties; in particular, both the sum and composition of two polynomials are always polynomials too.
4. The choice of this class has led to a development of an extensive theory called *computational complexity*, which seeks to classify problems according to their inherent difficulty.



P and NP Problems:

Class P is a class of decision problems that can be solved in polynomial time by (deterministic) algorithms. This class of problems is called **polynomial**.

The restriction of P to decision problems can be justified by the following reasons.

1. It is sensible to exclude problems not solvable in polynomial time because of their exponentially large output. Such problems do arise naturally.
2. Many important problems that are not decision problems in their most **natural formulation can be reduced to a series of decision problems that are easier** to study.

It is natural to wonder whether *every* decision problem can be solved in polynomial time. The answer to this question turns out to be no. In fact, some decision problems cannot be solved at all by any algorithm. Such problems are called *undecidable*. A famous example was given by Alan Turing in 1936. The problem in question is called the *halting problem*: given a computer program and an input to it, determine whether the program will halt on that input or continue working indefinitely on it.

Short proof of this remarkable fact. By way of contradiction, assume that A is an algorithm that solves the halting problem. That is, for any program P and input I

$$A(P, I) = \begin{cases} 1, & \text{if program } P \text{ halts on input } I; \\ 0, & \text{if program } P \text{ does not halt on input } I. \end{cases}$$

We can consider program P as an input to itself and use the output of algorithm A for pair (P, P) to construct a program Q as follows:

$$Q(P) = \begin{cases} \text{halts, if } A(P, P) = 0, \text{ i.e., if program } P \text{ does not halt on input } P; \\ \text{does not halt, if } A(P, P) = 1, \text{ i.e., if program } P \text{ halts on input } P. \end{cases}$$

Then on substituting Q for P , we obtain

$$Q(Q) = \begin{cases} \text{halts, if } A(Q, Q) = 0, \text{ i.e., if program } Q \text{ does not halt on input } Q; \\ \text{does not halt, if } A(Q, Q) = 1, \text{ i.e., if program } Q \text{ halts on input } Q. \end{cases}$$

This is a contradiction because neither of the two outcomes for program Q is possible, and it completes the proof.

Small sample of some of the best-known problems that fall into this category:

Hamiltonian circuit Determine whether a given graph has a Hamiltonian circuit (a path that starts and ends at the same vertex and passes through all the other vertices exactly once).

Traveling salesman Find the shortest tour through n cities with known positive integer distances between them (find the shortest Hamiltonian circuit in a complete graph with positive integer weights).

Knapsack problem Find the most valuable subset of n items of given positive integer weights and values that fit into a knapsack of a given positive integer capacity.

Partition problem Given n positive integers, determine whether it is possible to partition them into two disjoint subsets with the same sum.

Bin packing Given n items whose sizes are positive rational numbers not larger than 1, put them into the smallest number of bins of size 1.

Graph coloring For a given graph, find its chromatic number (the smallest number of colors that need to be assigned to the graph's vertices so that no two adjacent vertices are assigned the same color).

Integer linear programming Find the maximum (or minimum) value of a linear function of several integer-valued variables subject to a finite set of constraints in the form of linear equalities and/or inequalities.

Nondeterministic algorithm:

A *nondeterministic algorithm* is a two-stage procedure that takes as its input an instance I of a decision problem and does the following.

Nondeterministic ("guessing") stage: An arbitrary string S is generated that can be thought of as a candidate solution to the given instance I (but may be complete gibberish as well).

Deterministic ("verification") stage: A deterministic algorithm takes both I and S as its input and outputs yes if S represents a solution to instance I .

NP problems.

Class NP is the class of decision problems that can be solved by nondeterministic polynomial algorithms. This class of problems is called *nondeterministic polynomial*.

Most decision problems are in NP . First of all, this class includes all the problems in P :

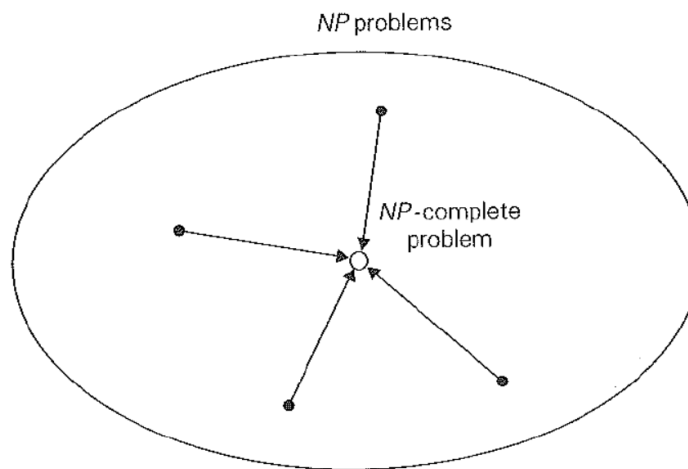
$$P \subseteq NP.$$

NP-Complete Problems:

NP-complete problem is a problem in NP that is as difficult as any other problem in this class because, by definition, any other problem in NP can be reduced to it in polynomial time.

A decision problem $D1$ is said to be *polynomially reducible* to a decision problem $D2$ if there exists a function t that transforms instances of $D1$ to instances of $D2$ such that

1. t maps all yes instances of $D1$ to yes instances of $D2$ and all no instances of $D1$ to no instances of $D2$;
2. t is computable by a polynomial-time algorithm.



Notion of an NP -complete problem. Polynomial-time reductions of NP problems to an NP -complete problem are shown by arrows.

A decision problem D is said to be NP -complete if

1. it belongs to class NP ;
2. every problem in NP is polynomially reducible to D ,

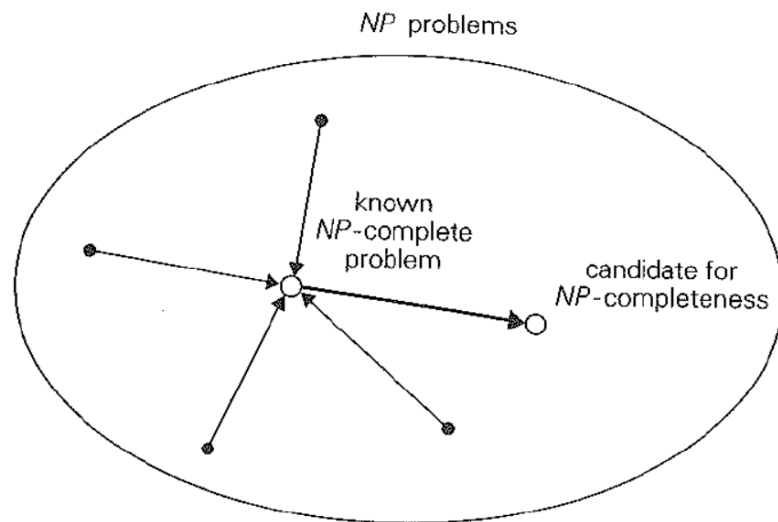
The notion of NP -completeness requires, however, polynomial reducibility of *all* problems in NP , both known and unknown, to the problem in question.

Given the bewildering variety of decision problems, it is nothing short of amazing that the specific examples of NP -complete problems have been actually found.

Showing that a decision problem is NP -complete can be done in two steps.

- 1) one needs to show that the problem in question is in NP ; i.e., a randomly generated string can be checked in polynomial time to determine whether or not it represents a solution to the problem. Typically, this step is easy.
- 2) To show that every problem in NP is reducible to the problem in question in polynomial time. Because of the transitivity of polynomial reduction, this step can be done by showing that a known NP -complete problem can be transformed to the problem in question in polynomial time

The definition of NP -completeness immediately implies that if there exists a deterministic polynomial-time algorithm for just one NP -complete problem, then every problem in NP can be solved in polynomial time by a deterministic algorithm, and hence $P = NP$.



Proving *NP*-completeness by reduction